Software Architecture and Design Specification

Project: College Timetable Generation System

Version: 1.0

Authors: SAATHVIK SAI SUVARNA-PES1UG24AM238

AYUSH PM -PES1UG24AM909

RISHI BHARADWAJ – PES1UG24AM223

ROHIT CHAKI – PES1UG24AM228

Date: 07-10-2026

Status: Baseline / Draft

Revision History

Version	Date	Author	Change Summary
1.0	07-10-2026	SAATHVIK SAI SUVARNA-PES1UG24AM238 AYUSH PM -PES1UG24AM909 RISHI BHARADWAJ – PES1UG24AM223 ROHIT CHAKI – PES1UG24AM228	Initial comprehensive architecture and design specification

Approvals

Role	Name	Signature/Date
Course Coordinator	Dr. Faculty Evaluator	evaluator@university.edu / 07-10-2026
System Architect	Project Team Lead	team@university.edu / 07-10-2026

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed system design for the College Timetable Generation System. It defines the high-level structural patterns,

component boundaries, data models, UML behavioral diagrams, API contracts, security threat models, and algorithmic workflows required to construct and evaluate the platform.

1.2 Scope

The scope encompasses the full software topology: the responsive single-page web interface, the RESTful application layer, the mathematical constraint optimization engine powered by Google OR-Tools (CP-SAT), relational database persistence, and document export subsystems. It details operational mechanics for scheduling theory and lab courses while enforcing zero hard conflicts. External student fee portals and central university ERPs are explicitly out of scope.

1.3 Audience

- Software Developers and Algorithm Engineers
- QA Engineers and System Testers
- System Administrators and DevOps Engineers
- Academic Instructors and Project Evaluators

1.4 Definitions

- **ADR:** Architecture Decision Record
- **CP-SAT:** Constraint Programming - Boolean Satisfiability (Google OR-Tools engine)
- **CSP:** Constraint Satisfaction Problem
- **DTO:** Data Transfer Object
- **JWT:** JSON Web Token
- **ORM:** Object-Relational Mapping
- **RBAC:** Role-Based Access Control
- **STRIDE:** Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

2. Document Overview

2.1 How to use this document

This specification serves as the technical blueprint for the engineering lifecycle. Section 3 presents the architectural decomposition, component diagrams, technology stack rationale, and STRIDE security framework. Section 4 provides design models, including complete UML sequence diagrams, formal REST API interface contracts, error propagation standards, and UX wireframe structures.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.0
- Software Test Plan (STP) v1.0
- Requirements Traceability Matrix (RTM) v1.0

3. Architecture

3.1 Goals & Constraints

Architectural Goals:

- **Zero Hard-Constraint Violations:** Mathematical guarantee that no faculty, room, or student section is double-booked.
- **High Responsiveness:** End-to-end solving time ≤ 15 seconds for up to 30 academic sections.
- **Operational Availability:** ≥ 99.5% uptime during peak semester scheduling windows.
- **Maintainability & Decoupling:** Independent isolation of the mathematical solver from web presentation logic.

Constraints:

- Open-source software dependencies without proprietary enterprise licenses.
- Standard horizontal deployment footprint executable within a 2 vCPU, 4 GB RAM container environment.
- Stateless REST API architecture to support horizontal scalability.

3.2 Stakeholders & Concerns

- **Academic Coordinators:** Dependability of constraint generation, intuitive manual override handling, instant conflict notifications.
- **Faculty Members:** Balanced daily teaching distribution, respect for leave and unavailable time slots.
- **Students:** Cohesive schedules free of concurrent lecture clashes and disjointed waiting gaps.
- **System Developers:** Separation of concerns, testability of optimization code, clear API documentation.

3.3 Component (UML) Diagram

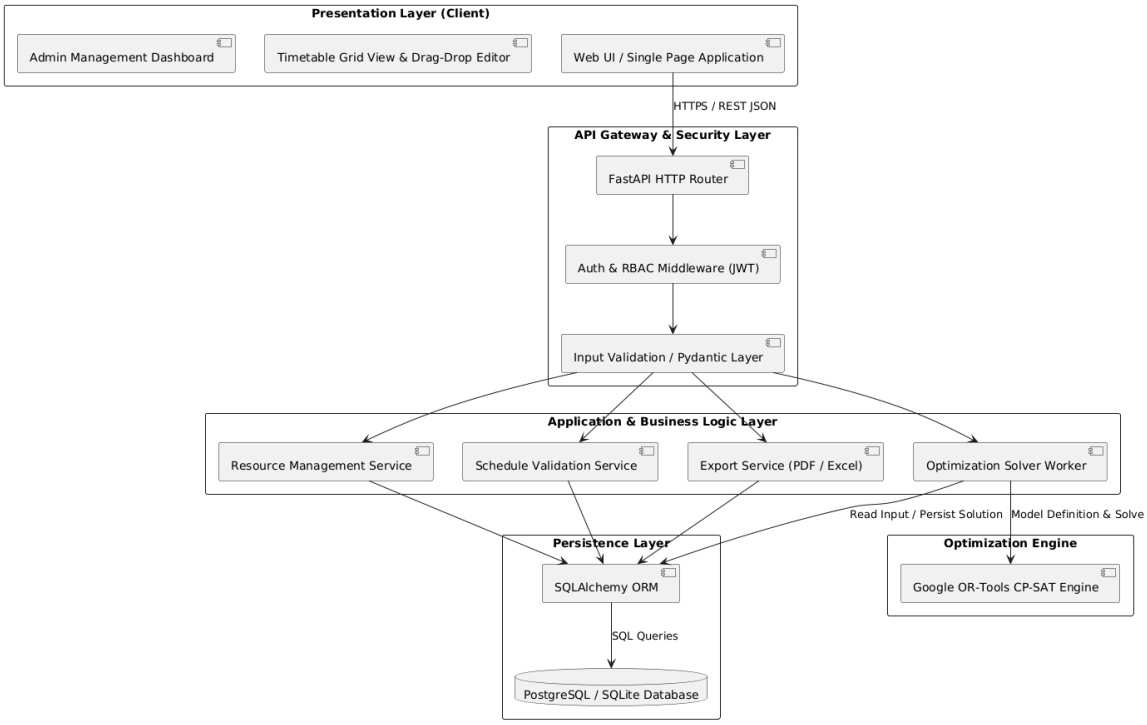


Figure 3.1: Component Diagram of the College Timetable Generation System

3.4 Component Descriptions

- **Web UI / Single Page Application:** Client-side user interface built with modern JavaScript/React and Tailwind CSS, presenting responsive management tables and interactive schedule grids.
- **FastAPI HTTP Router & Middleware:** Entry point handling asynchronous HTTP/REST requests, terminating TLS, enforcing JWT token validation, and sanitizing payloads via Pydantic models.
- **Resource Management Service:** Orchestrates domain logic for CRUD operations on Courses, Faculty, Classrooms, Laboratories, and Sections.
- **Schedule Validation Service:** Executes deterministic rule checks against proposed manual modifications, validating room capacities, instructor availability, and contiguous time slots before database commit.
- **Optimization Solver Worker:** Translates stored academic parameters into an integer constraint programming model, compiles variables, executes Google OR-Tools CP-SAT, and returns the solved timetable.
- **Export Service:** Formats finalized timetables into PDF layouts using ReportLab/WeasyPrint and multi-sheet Excel files using OpenPyXL.
- **Database & ORM:** Relational database schema interfaced via SQLAlchemy to guarantee ACID compliance and relational integrity across academic entities.

3.5 Chosen Architecture Pattern and Rationale

The system adopts a Layered Client-Server Architecture with an Asynchronous Decoupled Worker Pattern.

- **Rationale for Layered Architecture:** Isolates UI rendering from mathematical optimization. Changes to room algorithms or solver engines do not impact frontend views or database schemas.
- **Rationale for Asynchronous Solver Worker:** Timetable generation is CPU-bound. Running optimization within a synchronous HTTP request thread blocks the server event loop and triggers gateway timeouts (HTTP 504). Offloading optimization tasks to background worker routines allows the client to poll execution progress seamlessly.
- **Rejection of Distributed Microservices:** Deemed an unnecessary operational overhead for an academic institution scheduling system. A modular monolith with clear service layer boundaries provides high maintainability without network latency or container orchestration complexity.

3.6 Technology Stack & Data Stores

- **Frontend:** React 18, Tailwind CSS, Lucide Icons, HTML5 Drag and Drop API.
- **Backend API Framework:** Python 3.10+, FastAPI, Pydantic v2, Uvicorn ASGI Server.
- **Constraint Optimization Library:** Google OR-Tools (ortools.sat.python.cp_model).
- **Data Persistence:** PostgreSQL 15+ (Production) / SQLite 3 (Local testing and development), SQLAlchemy 2.0 ORM, Alembic (schema migrations).
- **Document Generation:** ReportLab (PDF generation) and OpenPyXL (Excel generation).
- **Security & Auth:** python-jose (JWT), passlib with BCrypt hashing.

3.7 Risks & Mitigations

Risk Description	Severity	Likelihood	Architectural Mitigation Strategy
NP-Hard Solver Divergence: Heavily over-constrained academic schedules cause infinite search loops.	High	Medium	Hard cap CP-SAT search duration using solver.parameters.max_time_in_seconds = 15.0. Return clear INFEASIBLE diagnostic feedback on time expiration.
Concurrent Manual Modification Clashes: Multiple coordinators adjust the same timetable simultaneously, overwriting slots.	High	Low	Implement optimistic concurrency control via a version_id column on the Timetable entity. Reject stale edits with HTTP 409 Conflict.
Memory Exhaustion During Search: Combinatorial explosion of Boolean variables exhausts server memory.	Medium	Low	Discretize slots into tightly bounded integer index tuples (section_id, course_id, room_id, timeslot_id). Prune impossible slots prior to model instantiation.

Risk Description	Severity	Likelihood	Architectural Mitigation Strategy
Relational Data Corruption: Cascade deletion of a faculty member or classroom deletes historical published timetables.	Medium	Medium	Implement soft-delete flags (is_active = false) and enforce strict foreign key constraints across all academic entity tables.

3.8 Traceability to Requirements

SRS Requirement ID	Functional Scope	Architectural Component	Design Specification Reference
TT-F-001 – TT-F-004	Master Resource CRUD	Resource Management Service	SAD Section 3.4 / DB Schema
TT-F-005 – TT-F-008	Hard Constraints Enforcement	Optimization Solver Engine	SAD Section 3.5 / CP-SAT Model
TT-F-009 – TT-F-011	Master Schedule Solving & Soft Rules	Optimization Solver Worker	SAD Section 4.2 / Sequence 1
TT-F-012	Multi-View Filterable Timetable Grid	Web UI (Timetable Grid Component)	SAD Section 4.5 / UX Design
TT-F-013 – TT-F-014	Manual Reassignment & Slot Pinning	Schedule Validation Service	SAD Section 4.2 / Sequence 2 & API 4.3
TT-F-015	PDF & Excel Export Generation	Export Service	SAD Section 3.4 / Document Module
TT-SR-001 – TT-SR-005	Security & Rate Limiting Controls	Auth Gateway & STRIDE Model	SAD Section 3.9 / Security Architecture

3.9 Security Architecture (STRIDE Threat Model)

STRIDE Category	Identified Threat	System Impact	Mitigation Mechanism
Spoofing	Attacker impersonates an Academic Coordinator to alter schedules.	Unauthorized schedule tampering.	Enforce signed JWT Bearer tokens containing user ID and verified role claims. Reject expired tokens.
Tampering	Malicious payload injection in room names or faculty codes.	Database injection, XSS attacks.	Strict Pydantic schema validation on all inputs; parameterized SQL queries via SQLAlchemy ORM.
Repudiation	Coordinator denies altering an approved timetable slot.	Disputed scheduling changes.	Immutable audit logging table recording user ID, target timetable ID, timestamp, and previous/new slot values.
Information Disclosure	Interception of confidential faculty leave or institutional schedules.	Breach of staff scheduling privacy.	Mandatory TLS 1.3 encryption across all communication endpoints; sensitive password hashing via BCrypt (cost ≥ 12).
Denial of Service	Flooding the generation API with concurrent solving requests.	Server CPU exhaustion, application hang.	Token bucket rate limiting (maximum 5 requests/min per IP) via middleware, coupled with asynchronous queue throttling.

STRIDE Category	Identified Threat	System Impact	Mitigation Mechanism
Elevation of Privilege	Student account attempts to invoke manual schedule override API.	Unauthorized timetable modification.	Role-Based Access Control (RBAC) enforced via FastAPI dependency injection guards on all mutating routes.

4. Design

4.1 Design Overview

The internal design enforces strict modular separation between presentation components, HTTP controllers, scheduling domain logic, and optimization solver routines. Data exchange between layers utilizes typed Data Transfer Objects (DTOs) modeled in Pydantic, preventing raw entity leaks and guaranteeing strict schema validation.

4.2 UML Sequence Diagrams

Flow 1: Automated Master Timetable Generation Flow

This diagram illustrates the asynchronous workflow where a Coordinator initiates a generation run, the CP-SAT engine computes the conflict-free assignment, and results are persisted.

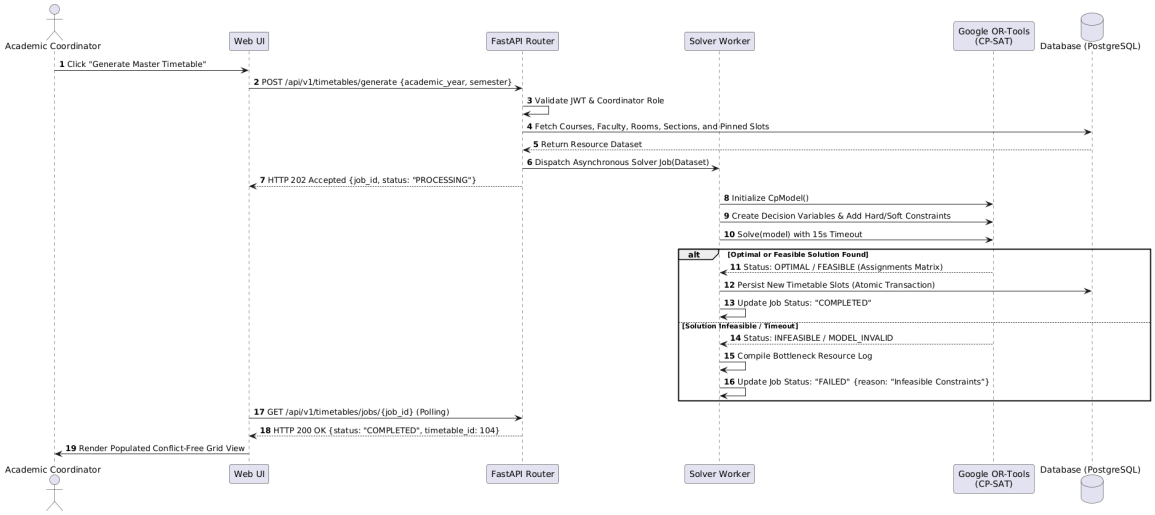


Figure 4.1: Sequence Diagram - Automated Master Timetable Generation

Flow 2: Manual Slot Drag-and-Drop Reassignment Flow

This diagram illustrates how manual coordinator edits are checked against hard constraints in real time before persistence.

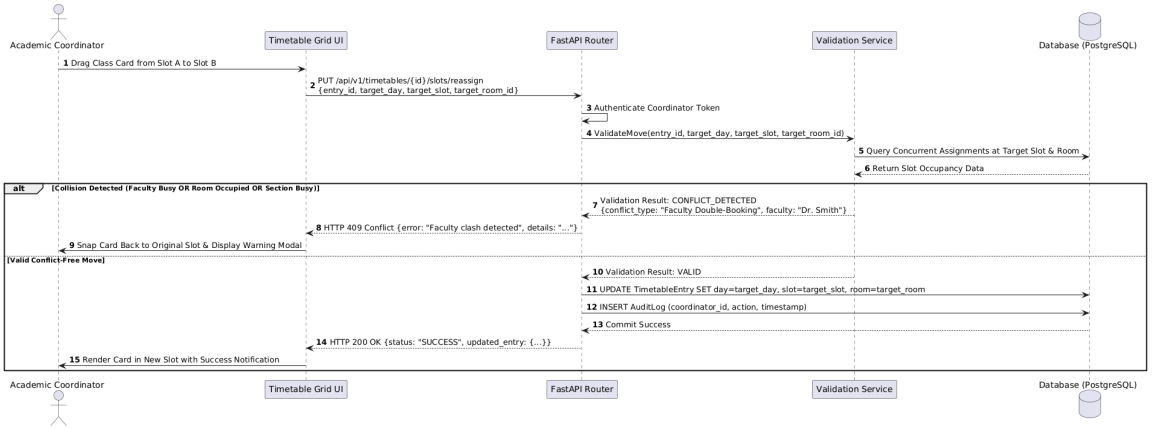


Figure 4.2: Sequence Diagram - Manual Slot Drag-and-Drop Reassignment

4.3 API Design

Endpoint 1: Initiate Automated Timetable Generation

Path: /api/v1/timetables/generate
Method: POST
Headers: Authorization: Bearer <JWT_TOKEN>, Content-Type: application/json
Description: Initiates an asynchronous background job to compile academic master data and execute the Google OR-Tools constraint solver.

Request Schema:

```
{
  "academic_year": "2026-2027",
  "semester": 5,
  "department_id": "CS",
  "solver_timeout_seconds": 15,
  "enforce_soft_constraints": true
}
```

Success Response (HTTP 202 Accepted):

```
{
  "job_id": "job_8f29e1c4",
  "status": "PROCESSING",
  "message": "Constraint satisfaction generation job queued successfully.",
  "estimated_completion_seconds": 10
}
```

Error Responses:

- **401 Unauthorized:** Missing or malformed JWT token.
- **403 Forbidden:** User does not hold the Academic Coordinator role.
- **422 Unprocessable Entity:** Invalid semester number or unrecognized department ID.

- **429 Too Many Requests:** Rate limit exceeded (more than 5 requests per minute).

Endpoint 2: Reassign Timetable Slot (Manual Modification)

Path: /api/v1/timetables/{timetable_id}/slots/reassign
Method: PUT
Headers: Authorization: Bearer <JWT_TOKEN>, Content-Type: application/json
Description: Validates and applies a manual drag-and-drop schedule edit. Rejects changes that introduce faculty, room, or section double-booking.

Request Schema:

```
{
  "entry_id": 412,
  "target_day": "Tuesday",
  "target_period_index": 3,
  "target_room_id": "LH-201"
}
```

Success Response (HTTP 200 OK):

```
{
  "status": "SUCCESS",
  "timetable_id": 104,
  "updated_entry": {
    "entry_id": 412,
    "course_code": "CS302",
    "faculty_id": "FAC_109",
    "section_id": "SEC_5A",
    "room_id": "LH-201",
    "day": "Tuesday",
    "period_index": 3
  },
  "message": "Slot reassignment validated and committed successfully."
}
```

Error Responses:

- **400 Bad Request:** Target room seating capacity is smaller than section strength.
- **401 Unauthorized:** Token invalid or expired.
- **409 Conflict:** Conflict detected (e.g., target faculty is already assigned to another class during period 3).

Example Error Response (HTTP 409 Conflict):

```
{
  "status": "ERROR",
  "error_code": "SCHEDULE_CONFLICT",
  "message": "Hard constraint violation: Faculty 'FAC_109' is already assigned to
'SEC_5C' in room 'LH-102' during Tuesday Period 3.",
  "conflict_type": "FACULTY_DOUBLE_BOOKING"
}
```

4.4 Error Handling, Logging & Monitoring

- **Error Propagation:** All API errors return a standardized RFC 7807 problem details JSON payload containing an error_code, human-readable message, and detailed field context.
- **Centralized Logging:** Handled via Python's standard logging library configured for structured JSON output. Logs capture HTTP verb, endpoint, execution latency, authenticated user ID, and unique trace_id. Sensitive credentials and personal faculty data are masked.
- **System Monitoring:** The application exposes a /health endpoint reporting database connectivity, memory utilization, and active background solver task counts.

4.5 UX Design

Master Grid Layout: Displays a two-dimensional weekly matrix with Days (Monday through Friday/Saturday) along the vertical Y-axis and standard Academic Periods (e.g., 08:30–09:30 through 16:30–17:30) along the horizontal X-axis.

Interactive Elements:

- Class cards display Subject Code, Subject Name, Faculty Initials, and Allocated Room.
- Theory sessions appear as single-slot cards, while Practical Labs span 2 or 3 contiguous slot blocks.
- Drag-and-drop visual indicators: Hovering over an available slot highlights the border green; hovering over an invalid clashing slot displays a red lock badge.

Accessibility: High-contrast color assignments, screen-reader-compliant semantic table elements, and full keyboard navigation in compliance with WCAG 2.1 AA.

4.6 Open Issues & Next Steps

- **Multi-Campus Transit Modeling:** Current version assumes all rooms reside on a single contiguous campus. Future versions will incorporate physical transit time penalties between distant campus blocks.
- **Elective Course Splitting:** Managing open-elective baskets where multiple subjects run simultaneously across splintered student groups will be integrated into the next CP-SAT model iteration.

5. Appendices

5.1 Glossary

- **Constraint Satisfaction Problem (CSP):** Mathematical problem defined as a set of variables that must take values satisfying a set of constraints.
- **Google OR-Tools CP-SAT:** An award-winning open-source software suite for combinatorial optimization that solves constraint programming models via SAT techniques.

- **Hard Constraint:** A mandatory rule that must not be broken (e.g., no instructor in two places at once).
- **Soft Constraint:** A preferred scheduling condition aimed at optimizing comfort or efficiency (e.g., avoiding afternoon teaching gaps).
- **Time Discretization:** The breakdown of an academic day into discrete, indexable periods.

5.2 References

- IEEE Standard 42010-2011: Systems and Software Engineering — Architecture Description.
- Google Developers: OR-Tools CP-SAT Solver Documentation & Best Practices (2025/2026).
- OWASP Foundation: OWASP Top 10 Security Vulnerabilities and Mitigation Standards.
- Martin Fowler: Patterns of Enterprise Application Architecture.

5.3 Tools

- **Modeling & Design:** PlantUML, diagrams.net (draw.io)
- **API Documentation & Testing:** Swagger UI (OpenAPI 3.0), Postman
- **Development Environment:** Python 3.10+, VS Code / PyCharm, Docker